

System Programming Project 4

담당 교수 : 이영민 교수님

이름 : 변정현

학번 : 20210054

1. 개발 구조(Design Overview)

segregated free list 기반의 메모리 할당기를 구현해서 서로 다른 크기의 블록을 크기별로 버킷(bucket)에 나눠서 관리함으로써 빠르게 탐색하고 할당할 수 있다. 전체 힙은 mem_sbrk()를 통해 확장하고, 크기 기준으로 20개의 free list를 사용한다. 각 블록은 header, footer, prev/next 포인터를 포함한 이중 연결 리스트로 구성되어 있다.

Flow Chart:

1. 요청이 malloc/realloc인 경우:

[요청: malloc, realloc]

↓

[get_seglist_index(size)]

↓

[search_fit_block(size) → 적절한 free 블록 탐색]

→ 못 찾으면 extend_heap()으로 힙 확장

↓

[place(bp, size) → 할당 대상 블록의 header/footer를 할당 상태로 세팅

+ 필요 시 블록을 분할(split)해서 나머지는 free로 남기고 free list에 삽입]

↓

[할당된 블록의 시작 포인터 반환]

2. 요청이 free인 경우

[요청: free]

↓

[header/footer의 할당 비트를 0으로 변경]

↓

[coalesce(bp) → 인접한 free 블록과 병합]

→ 내부에서 insert_free_block() 호출

↓

[병합된 free block을 적절한 seglist에 삽입]

2. 주요 흐름

mm_init(): segFList 공간 확보 후 NULL 초기화. 그 다음 heap_listp 초기화 및 extend_heap으로 첫 가용 블록 확보.

malloc(size_t size): 크기 정렬 후 적절한 리스트에서 블록 탐색. 없으면 extend_heap(). 있다면 place()로 할당.

free(void *bp): 블록을 반환하고 coalesce()를 통해 병합 후, free list에 삽입.

realloc(void *ptr, size_t size): 새 블록을 malloc 후 기존 데이터 복사, 기존 블록 free.

coalesce(void *bp): 이전/다음 블록이 free 상태일 경우 병합. 병합된 블록은 다시 insert.

get_seglist_index(size_t size): 블록 크기에 따라 seglist 인덱스 결정. MINSIZE 기준으로 2배씩 커지는 방식.

insert_free_block(void *bp, size_t size): 적절한 리스트에 포인터 순서 맞춰 삽입. prev/next 연결.

remove_from_free_list(void *bp): 해당 블록을 리스트에서 제거. 연결 포인터 정리.

search_fit_block(size_t size): 해당 크기 이상인 블록을 first-fit으로 탐색.

3. Global Variables and Structs

segFList (void**): 크기별 블록 리스트를 가리키는 포인터 배열. 각 리스트는 이중 연결 리스트.

heap_listp (char*): 실제 할당 블록이 시작되는 기준 포인터.

4. 부가적 흐름

coalesce(bp): 이전, 다음 블록이 가용 상태인지 확인하고 병합. 병합된 블록은 remove_from_free_list() 후 insert.

place(bp, asize): 블록을 할당할 위치에 실제로 넣는 함수. 남는 부분이 있으면 split 해서 나머지를 가용 블록으로 설정.

extend_heap(words): 힙을 일정 단위(words * 4bytes)만큼 확장하고, 새로 만들어진 가용 블록을

coalesce 후 삽입.

get_seglist_index(size): size보다 크거나 같은 최소 범위를 가지는 인덱스를 반환. 범위는 16, 32, 64, ..., 2^n 식으로 증가.